



Functions

Chapter 04

Introduction

- A function is a block of code that performs a specific task.
- A function can be called multiple times throughout a program.
- A function is designed to make code more reusable, organized, and easier to maintain by encapsulating a set of instructions with a clear name and purpose.
- Essentially, a function is a self-contained unit that takes input data, processes it, and returns a result when called.

4.2 Defining Functions

- `square` function that calculates the square of its argument.

```
In [1]: def square(number):  
        """Calculate the square of number."""  
        return number ** 2
```

```
In [2]: square(7)
```

```
Out[2]: 49
```

```
In [3]: square(2.5)
```

```
Out[3]: 6.25
```

- Calling `square` with a non-numeric argument like `'hello'` causes a `TypeError` because the exponentiation operator (`**`) works only with numeric values

Defining a Custom Function

- Definition begins with the (`def` keyword, followed by the function name, a set of parentheses and a colon (`:`).
- By convention function names should begin with a lowercase letter and in multiword names underscores should separate each word.
- Required parentheses contain the function's **parameter list**.
- Empty parentheses mean no parameters.
- The indented lines after the colon (`:`) are the function's **block**
 - Consists of an optional docstring followed by the statements that perform the function's task.

Returning a Result to a Function's Caller

- Function calls also can be embedded in expressions:

```
In [4]: print('The square of 7 is', square(7))
```

The square of 7 is 49

- Three Ways to Return a Result to a Function's Caller
 - `return` followed by an expression.
 - `return` without an expression implicitly returns `None` —represents the **absence of a value** and **evaluates to `False` in conditions**.
 - **No `return` statement implicitly returns `None`.**

Accessing a Function's Docstring Via IPython's Help Mechanism

- Following a function's name with `?` in IPython displays its docstring:

In [5]: `square?`

```
Signature: square(number)
Docstring: Calculate the square of number.
File:      ~/Dropbox/books/2019/Python/PyCDS_JupyterSlides/ch04/<ipython-input-1-7d5dc51751d0>
Type:      function
```

- If the function's source code is accessible from IPython, `??` displays the function's docstring and full source-code definition:

In [6]: `square??`

```
Signature: square(number)
Source:
def square(number):
    """Calculate the square of number."""
    return number ** 2
File:      ~/Dropbox/books/2019/Python/PyCDS_JupyterSlides/ch04/<ipython-input-1-7d5dc51751d0>
Type:      function
```

4.3 Functions with Multiple Parameters

- `maximum` function that determines and returns the largest of three values.

```
In [1]: def maximum(value1, value2, value3):  
        """Return the maximum of three values."""  
        max_value = value1  
        if value2 > max_value:  
            max_value = value2  
        if value3 > max_value:  
            max_value = value3  
        return max_value
```

```
In [2]: maximum(12, 27, 36)
```

```
Out[2]: 36
```

```
In [3]: maximum(12.3, 45.6, 9.7)
```

```
Out[3]: 45.6
```

```
In [4]: maximum('yellow', 'red', 'orange')
```

```
Out[4]: 'yellow'
```

```
In [5]: maximum(13.5, -3, 7)
```

```
Out[5]: 13.5
```

Function maximum's Definition

- Specify multiple parameters in a comma-separated list.
- To determine the largest value:
 - Assume that `value1` contains the largest value.
 - The first `if` statement then tests `value2 > max_value`, and if this condition is `True` assigns `value2` to `max_value`.
 - The second `if` statement then tests `value3 > max_value`, and if this condition is `True` assigns `value3` to `max_value`.
- Now, `max_value` contains the largest value.

Python's Built-In max and min Functions

- For many common tasks, the capabilities you need already exist in Python.
- Built-in `max` and `min` functions know how to determine the largest and smallest of their two or more arguments:

```
In [6]: max('yellow', 'red', 'orange', 'blue', 'green')
```

```
Out[6]: 'yellow'
```

```
In [7]: min(15, 9, 27, 14)
```

```
Out[7]: 9
```

- Each also can receive an iterable argument, such as a list or a string.
- Using built-in functions or functions from the Python Standard Library's modules rather than writing your own can reduce development time and increase program reliability, portability and performance.

4.4 Random-Number Generation

Rolling a Six-Sided Die

- Product 10 random integers in the range 1–6 to simulate rolling a six-sided die:

```
In [1]: import random
```

```
In [2]: for roll in range(10):  
        print(random.randrange(1, 7), end=' ')
```

6 2 3 5 6 5 1 1 3 2

- `randrange` function generates an integer from the first argument value up to, but not including, the second argument value.
- Different values are displayed if you re-execute the loop.

```
In [3]: for roll in range(10):  
        print(random.randrange(1, 7), end=' ')
```

3 3 4 2 6 2 4 3 5 6

- Can guarantee **reproducibility** of a random sequence with the `random` module's `seed` function.

Rolling a Six-Sided Die 6,000,000 Times

- If `randrange` truly produces integers at random, every number in its range has an equal probability (or chance or likelihood) of being returned each time we call it.
- Roll a die 6,000,000 times.
- Each die face should occur approximately 1,000,000 times.
- We used Python's underscore (`_`) digit separator to make the value 6000000 more readable.

```
### Rolling a Six-Sided Die 6,000,000 Times

# fig04_01.py
"""Roll a six-sided die 6,000,000 times."""
import random

# face frequency counters
frequency1 = 0
frequency2 = 0
frequency3 = 0
frequency4 = 0
frequency5 = 0
frequency6 = 0
```

```

# 6,000,000 die rolls
for roll in range(6_000_000): # note underscore separators
    face = random.randrange(1, 7)

    # increment appropriate face counter
    if face == 1:
        frequency1 += 1
    elif face == 2:
        frequency2 += 1
    elif face == 3:
        frequency3 += 1
    elif face == 4:
        frequency4 += 1
    elif face == 5:
        frequency5 += 1
    elif face == 6:
        frequency6 += 1

print(f'Face{"Frequency":>13}')
print(f'{1:>4}{frequency1:>13}')
print(f'{2:>4}{frequency2:>13}')
print(f'{3:>4}{frequency3:>13}')
print(f'{4:>4}{frequency4:>13}')
print(f'{5:>4}{frequency5:>13}')
print(f'{6:>4}{frequency6:>13}')

```

In [4]: run fig04_01.py

Face	Frequency
1	1000562
2	999042
3	999988
4	1000966
5	999281
6	1000161

Seeding the Random-Number Generator for Reproducibility

- Function `randrange` generates pseudorandom numbers.
- Numbers appear to be random, because each time you start a new interactive session or execute a script that uses the random module's functions, Python internally uses a different seed value.
- When you're debugging logic errors in programs that use randomly generated data, it can be helpful to use the same sequence of random numbers.
- To do this, use the random module's `seed` function to seed the random-number generator:

```
In [5]: random.seed(32)
```

```
In [6]: for roll in range(10):  
        print(random.randrange(1, 7), end=' ')
```

```
1 2 2 3 6 2 4 1 6 1
```

```
In [7]: for roll in range(10):  
        print(random.randrange(1, 7), end=' ')
```

```
1 3 5 3 1 5 6 4 3 5
```

```
In [8]: random.seed(32)
```

```
In [9]: for roll in range(10):  
        print(random.randrange(1, 7), end=' ')
```

```
1 2 2 3 6 2 4 1 6 1
```

4.5 Case Study: A Game of Chance

- Requirements statement:

You roll two six-sided dice, each with faces containing one, two, three, four, five and six spots, respectively. When the dice come to rest, the sum of the spots on the two upward faces is calculated. If the sum is 7 or 11 on the first roll, you win. If the sum is 2, 3 or 12 on the first roll (called “craps”), you lose (i.e., the “house” wins). If the sum is 4, 5, 6, 8, 9 or 10 on the first roll, that sum becomes your “point.” To win, you must continue rolling the dice until you “make your point” (i.e., roll that same point value). You lose by rolling a 7 before making your point.

```
# fig04_02.py
"""Simulating the dice game Craps."""
import random

def roll_dice():
    """Roll two dice and return their face values as a tuple."""
    die1 = random.randrange(1, 7)
    die2 = random.randrange(1, 7)
    return (die1, die2) # pack die face values into a tuple

def display_dice(dice):
    """Display one roll of the two dice."""
    die1, die2 = dice # unpack the tuple into variables die1 and die2
    print(f'Player rolled {die1} + {die2} = {sum(dice)}')

die_values = roll_dice() # first roll
display_dice(die_values)
```

```

# determine game status and point, based on first roll
sum_of_dice = sum(die_values)

if sum_of_dice in (7, 11): # win
    game_status = 'WON'
elif sum_of_dice in (2, 3, 12): # lose
    game_status = 'LOST'
else: # remember point
    game_status = 'CONTINUE'
    my_point = sum_of_dice
    print('Point is', my_point)

# continue rolling until player wins or loses
while game_status == 'CONTINUE':
    die_values = roll_dice()
    display_dice(die_values)
    sum_of_dice = sum(die_values)

    if sum_of_dice == my_point: # win by making point
        game_status = 'WON'
    elif sum_of_dice == 7: # lose by rolling 7
        game_status = 'LOST'

# display "wins" or "loses" message
if game_status == 'WON':
    print('Player wins')
else:
    print('Player loses')

```

```
In [1]: run fig04_02.py
```

```
Player rolled 5 + 4 = 9  
Point is 9  
Player rolled 6 + 3 = 9  
Player wins
```

```
In [2]: run fig04_02.py
```

```
Player rolled 6 + 6 = 12  
Player loses
```

```
In [3]: run fig04_02.py
```

```
Player rolled 4 + 3 = 7  
Player wins
```

```
In [5]: run fig04_02.py
```

```
Player rolled 1 + 5 = 6  
Point is 6  
Player rolled 5 + 4 = 9  
Player rolled 6 + 3 = 9  
Player rolled 1 + 6 = 7  
Player loses
```

Function `roll_dice`—Returning Multiple Values Via a Tuple

- Simulates rolling two dice on each roll.
- Sometimes it's useful to return more than one value, as in `roll_dice`, which returns both die values as a **tuple**—an immutable sequences of values.
- To create a tuple, separate its values with commas—known as **packing a tuple**.
- Parentheses are optional, but we recommend using them for clarity.

Function `display_dice`

- Assigning a tuple to a comma-separated list of variables **unpacks the tuple**.
- Number of variables to the left of `=` must match the number of elements in the tuple; otherwise, a `ValueError` occurs.

First Roll

- When the script begins executing, we roll the dice and display the results.
- You can win or lose on the first roll or any subsequent roll—`game_status` keeps track of the win/loss status.
- The `in` operator in the following expression tests whether the tuple `(7, 11)` contains `sum_of_dice`'s value

```
sum_of_dice in (7, 11)
```

- The operator's right operand can be any iterable.
- The preceding concise condition is equivalent to

```
(sum_of_dice == 7) or (sum_of_dice == 11)
```

4.7 `math` Module Functions

- The `math` module defines functions for performing various common mathematical calculations.

```
In [1]: import math
```

```
In [2]: math.sqrt(900)
```

```
Out[2]: 30.0
```

```
In [3]: math.fabs(-10)
```

```
Out[3]: 10.0
```

- Some `math` module functions are summarized below

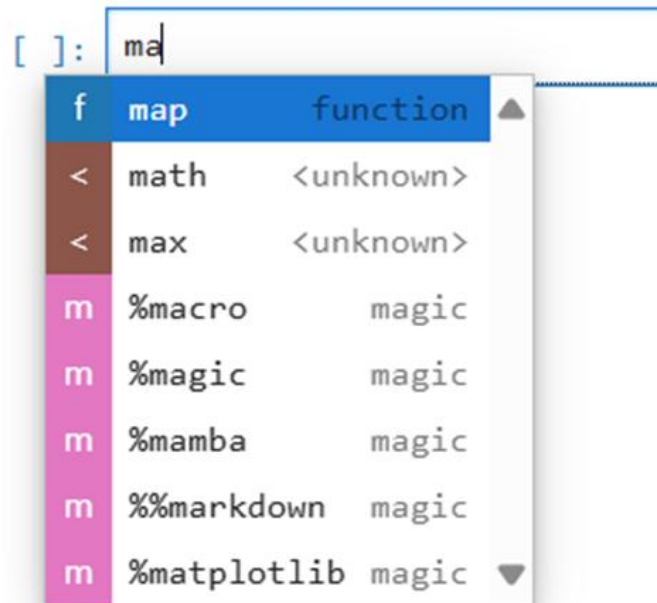
Function	Description	Example
<code>ceil(x)</code>	Rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 , <code>ceil(-9.8)</code> is -9.0
<code>floor(x)</code>	Rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 , <code>floor(-9.8)</code> is -10.0
<code>sin(x)</code>	Trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	Trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	Trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0
<code>exp(x)</code>	Exponential function e^x	<code>exp(1.0)</code> is 2.718282 , <code>exp(2.0)</code> is 7.389056
<code>log(x)</code>	Natural logarithm of x (base e)	<code>log(2.718282)</code> is 1.0 , <code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	Logarithm of x (base 10)	<code>log10(10.0)</code> is 1.0 , <code>log10(100.0)</code> is 2.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2.0, 7.0)</code> is 128.0 , <code>pow(9.0, .5)</code> is 3.0
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0 , <code>sqrt(9.0)</code> is 3.0
<code>fabs(x)</code>	Absolute value of x—always returns a float. Python also has the built-in function <code>abs</code> , which returns an <code>int</code> or a <code>float</code> , based on its argument.	<code>fabs(5.1)</code> is 5.1 , <code>fabs(-5.1)</code> is 5.1
<code>fmod(x, y)</code>	Remainder of x/y as a floating-point number	<code>fmod(9.8, 4.0)</code> is 1.8

4.8 Using IPython Tab Completion for Discovery

- View a module's documentation in IPython interactive mode via **tab completion**.
- After you type a portion of an identifier and press *Tab*, IPython completes the identifier for you or provides a list of identifiers that begin with what you've typed so far.

```
In [1]: import math
```

```
In [ ]: # press <Tab> after ma  
ma
```



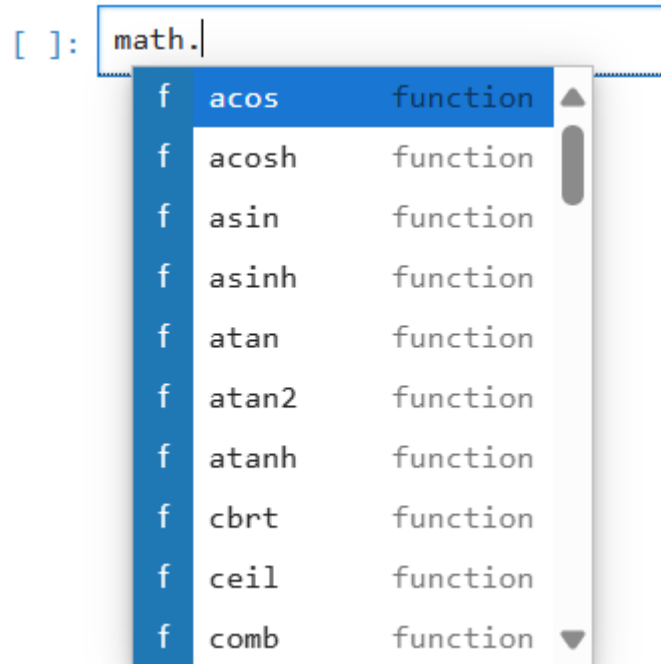
```
In [1]: import math  
In [2]: ma  
        map()    max    %magic    %%markdown  
        math     %macro %mamba    %matplotlib
```

Viewing Identifiers in a Module

- To view a list of identifiers defined in a module, type the module's name and a dot (`.`), then press *Tab*:

```
In [ ]: # press <Tab> after the dot  
math.
```

- Python does not have *constants*, although many objects in Python are immutable (nonmodifiable).
- Even though `pi` and `e` are real-world constants, **you must not assign new values to them**, because that would change their values.
- To help distinguish constants from other variables, the style guide recommends naming your custom constants with all capital letters.



```
In [2]: math.  
acos()  atan()  ceil()   cosh()   erf()  
acosh() atan2() comb()   degrees() erfc()  
asin()  atanh() copysign() dist()  exp()  
asinh() cbrt()  cos()    e       exp2()
```

4.9 Default Parameter Values

- You can specify that a parameter has a **default parameter value**.
- When calling the function, if you omit the argument for a parameter with a default parameter value, the default value for that parameter is automatically passed.

```
In [1]: def rectangle_area(length=2, width=3):  
        """Return a rectangle's area."""  
        return length * width
```

- Specify a default parameter value by following a parameter's name with an `=` and a value.
- Any parameters with default parameter values must appear in the parameter list to the *right* of parameters that do not have defaults.

```
In [2]: rectangle_area()
```

```
Out[2]: 6
```

```
In [3]: rectangle_area(10)
```

```
Out[3]: 30
```

```
In [4]: rectangle_area(10, 5)
```

```
Out[4]: 50
```

4.10 Keyword Arguments

- When calling functions, you can use **keyword arguments** to pass arguments in **any** order.

```
In [1]: def rectangle_area(length, width):  
        """Return a rectangle's area."""  
        return length * width
```

- Each keyword *argument in a call* has the form *parametername=value*.
- Order of keyword arguments does not matter.

```
In [2]: rectangle_area(width=5, length=10)
```

```
Out[2]: 50
```

4.11 Arbitrary Argument Lists

- Functions with **arbitrary argument lists**, such as built-in functions `min` and `max`, can receive *any* number of arguments.
- Function `min`'s documentation states that `min` has two *required* parameters (named `arg1` and `arg2`) and an optional third parameter of the form `*args`, indicating that the function can receive any number of additional arguments.
- The `*` before the parameter name tells Python to pack any remaining arguments into a tuple that's passed to the `args` parameter.

Defining a Function with an Arbitrary Argument List

- `average` function that can receive any number of arguments.

```
In [1]: def average(*args):  
        return sum(args) / len(args)
```

- The `*args` parameter must be the *rightmost* parameter.

```
In [2]: average(5, 10)
```

```
Out[2]: 7.5
```

```
In [3]: average(5, 10, 15)
```

```
Out[3]: 10.0
```

```
In [4]: average(5, 10, 15, 20)
```

```
Out[4]: 12.5
```

Passing an Iterable's Individual Elements as Function Arguments

- Can unpack a tuple's, list's or other iterable's elements to pass them as individual function arguments.
- The *** operator**, when applied to an iterable argument in a function call, unpacks its elements.

```
In [5]: grades = [88, 75, 96, 55, 83]
```

```
In [6]: average(*grades)
```

```
Out[6]: 79.4
```

- Equivalent to `average(88, 75, 96, 55, 83)`.

4.12 Methods: Functions That Belong to Objects

- A **method** is simply a function that you call on an object using the form

`object_name.method_name(arguments)`

```
In [1]: s = 'Hello'
```

```
In [2]: s.lower() # call lower method on string object s
```

```
Out[2]: 'hello'
```

```
In [3]: s.upper()
```

```
Out[3]: 'HELLO'
```

```
In [4]: s
```

```
Out[4]: 'Hello'
```


4.13 Scope Rules

- Each identifier has a `scope` that determines where you can use it in your program.

Local Scope

- A local variable's identifier has **local scope**.

Global Scope

- Identifiers defined outside any function (or class) have **global scope**—these may include functions, variables and classes.

Accessing a Global Variable from a Function

```
In [1]: x = 7
```

```
In [2]: def access_global():  
        print('x printed from access_global:', x)
```

```
In [3]: access_global()
```

```
x printed from access_global: 7
```

- By default, you cannot *modify* a global variable in a function
- Python creates a **new local variable** when you first assign a value to a variable in a function's block.
- In function `try_to_modify_global`'s block, the local `x` **shadows** the global `x`, making it inaccessible in the scope of the function's block.

```
In [4]: def try_to_modify_global():  
        x = 3.5  
        print('x printed from try_to_modify_global:', x)
```

```
In [5]: try_to_modify_global()  
  
x printed from try_to_modify_global: 3.5
```

```
In [6]: x
```

```
Out[6]: 7
```

- To modify a global variable in a function's block, you must use a **global** statement to declare that the variable is defined in the global scope:

```
In [7]: def modify_global():  
        global x;  
        x = 'hello'  
        print('x printed from modify_global:', x)
```

```
In [8]: modify_global()  
  
x printed from modify_global: hello
```

```
In [9]: x
```

```
Out[9]: 'hello'
```

Shadowing Functions

- In the preceding chapters, when summing values, we stored the sum in a variable named `total`.
- If you define a variable named `sum`, it *shadows* the built-in function `sum`, making it inaccessible in your code.

```
In [10]: sum = 10 + 5
```

```
In [11]: sum
```

```
Out[11]: 15
```

```
In [12]: sum([10, 5])
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-12-1237d97a65fb> in <module>  
----> 1 sum([10, 5])  
  
TypeError: 'int' object is not callable
```

4.14 `import`: A Deeper Look

Importing Multiple Identifiers from a Module

- Use `from...import` to import a comma-separated list of identifiers from a module then use them without having to precede them with the module name and a dot (`.`):

```
In [1]: from math import ceil, floor
```

```
In [2]: ceil(10.3)
```

```
Out[2]: 11
```

```
In [3]: floor(10.7)
```

```
Out[3]: 10
```

Caution: Avoid Wildcard Imports

- Import *all* identifiers defined in a module with a **wildcard** `import` .
- Makes all of the module's identifiers available.
- Can lead to subtle errors.
- Considered a dangerous practice.

```
In [4]: e = 'hello'
```

```
In [5]: from math import *
```

```
In [6]: e
```

```
Out[6]: 2.718281828459045
```

- After executing the import, variable `e` is replaced, possibly by accident, with the `math` module's constant `e` .

Binding Names for Modules and Module Identifiers

- Sometimes it's helpful to import a module and use an abbreviation for it to simplify your code.
- The `import` statement's `as` clause allows you to specify the name used to reference the module's identifiers.

```
In [7]: import statistics as stats
```

```
In [8]: grades = [85, 93, 45, 87, 93]
```

```
In [9]: stats.mean(grades)
```

```
Out[9]: 80.6
```

4.15 Passing Arguments to Functions: A Deeper Look

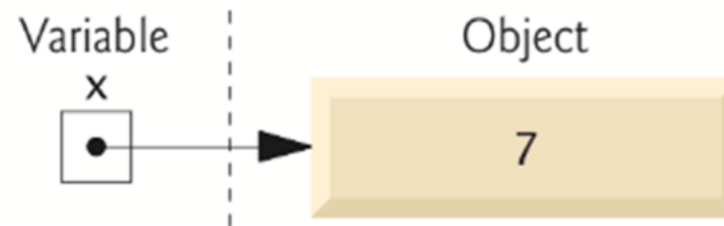
- Python arguments are always passed by reference.
- Some people call this **pass-by-object-reference**, because “everything in Python is an object.”
- When a function call provides an argument, Python copies the argument object’s *reference*—not the object itself—into the corresponding parameter.

Memory Addresses, References and “Pointers”

- After an assignment like the following,

```
In [1]: x = 7
```

the variable `x` contains a reference to an *object* containing `7` stored *elsewhere* in memory.



Built-In Function `id` and Object Identities

- Every object has a **unique identity**—an `int` value which **identifies only that object** while it remains in memory.
- **Built-in `id` function** to obtain an object's identity.

```
In [1]: x = 7
```

```
In [2]: id(x)
```

```
Out[2]: 4311725472
```

▼ Passing an Object to a Function

```
In [3]: def cube(number):  
        print('id(number):', id(number))  
        return number ** 3
```

```
In [4]: cube(x)
```

```
id(number): 4311725472
```

```
Out[4]: 343
```

- The identity displayed for `cube`'s parameter `number` is the *same* as that displayed for `x` previously.
- The *argument* `x` and the *parameter* `number` refer to the *same object* while `cube` executes.

4.18 Intro to Data Science: Measures of Dispersion

- Considered the measures of central tendency—mean, median and mode.
- Help us categorize typical values in a group.
- An entire group is called a **population**.
- Sometimes a population is quite large, such as the people likely to vote in the next U.S. presidential election, which is a number in excess of 100,000,000 people.
- For practical reasons, the polling organizations trying to predict who will become the next president work with carefully selected small subsets of the population known as **samples**.
- Here we introduce **measures of dispersion** (also called **measures of variability**) that help you understand how **spread out** the values are.
- We'll calculate each measure of dispersion both by hand and with functions from the module `statistics`, using the following population of 10 six-sided die rolls:

```
1, 3, 4, 2, 6, 5, 3, 4, 5, 2
```

Variance

- To determine variance, begin with the mean of these values—3.5.
- Next, subtract the mean from every die value:

```
-2.5, -0.5, 0.5, -1.5, 2.5, 1.5, -0.5, 0.5, 1.5, -1.5
```

- Then, square each of these results (yielding only positives):

```
6.25, 0.25, 0.25, 2.25, 6.25, 2.25, 0.25, 0.25, 2.25, 2.25
```

- Finally, calculate the mean of these squares, which is 2.25 (22.5 / 10)—this is the **population variance**.
- Squaring the difference between each die value and the mean of all die values emphasizes **outliers**—the values that are farthest from the mean—which can be important in data analysis.
- The following code uses the `statistics` module's `pvariance` function to confirm our manual result:

```
In [1]: import statistics
```

```
In [2]: statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
```

```
Out[2]: 2.25
```

$$\text{(Mean)} \mu = \frac{\sum_{i=1}^N x_i}{N}$$

$$\text{(Variance)} \sigma^2 = \frac{\sum_{i=1}^N (x_i - \mu)^2}{N}$$

$$\text{(Standard Deviation)} \sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N}}$$

Standard Deviation

- The standard deviation is the square root of the variance (in this case, 1.5), which tones down the effect of the outliers.
- The smaller the variance and standard deviation are, the closer the data values are to the mean and the less overall dispersion (that is, spread) there is between the values and the mean.
- The following code calculates the population standard deviation with the `statistics` module's `pstdev` function, confirming our manual result:

```
In [3]: statistics.pstdev([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
```

```
Out[3]: 1.5
```

```
In [4]: import math
```

```
In [5]: math.sqrt(statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2]))
```

```
Out[5]: 1.5
```